

VSecNN: Verifiable and Privacy-Preserving Neural Network Inference in Cloud Service

Wenti Yang¹, Xuan Li², Meng Li³, Senior Member, IEEE, Zijian Zhang⁴, Senior Member, IEEE, Zhitao Guan⁵, Member, IEEE, and Liehuang Zhu⁶, Senior Member, IEEE

Abstract—Neural network inference in cloud service offers tangible benefits to users, from individuals and small institutions to large companies. However, two crucial concerns must be addressed. The first arises in satisfying the privacy of the model, the input data, and the inference results throughout the inference process. The second pertains to verifying that the inferences are derived from the designated neural network model. Although Secure Multi-Party Computation (MPC) and Zero-Knowledge Proof (ZKP) are typically adopted to mitigate such issues, the major challenge lies in achieving privacy preservation and verifiability simultaneously. In this study, we address both issues by proposing VSecNN, a verifiable and privacy-preserving neural network inference scheme. Specifically, we integrate MPC with the Zero-Knowledge Succinct Non-Interactive Argument of Knowledge (zk-SNARK) protocol to achieve zero-knowledge proof generation for multiple parties. Subsequently, we perform adaptive optimizations on the multi-party proof generation approach to align with the neural network, thereby achieving both privacy-preserving capabilities and verifiability. Experimental results demonstrate an improvement in the efficiency. For example, the computation time for completing our multi-party proof generation could be as low as 1.7 times that of the single-party proof generation, while the verification requires only 169ms on the MNIST dataset.

Index Terms—Neural network inference, privacy preservation, verifiability, secure multi-party computation, zk-SNARK.

Received 30 March 2025; revised 16 August 2025; accepted 24 September 2025. Date of publication 1 October 2025; date of current version 9 October 2025. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62372173, Grant 62372149, Grant U23A20303, and Grant 62572168; in part by Anhui Provincial Natural Science Foundation under Grant 2508085MF151; in part by the Key Laboratory of Knowledge Engineering with Big Data (the Ministry of Education of China) under Grant BigKEOpen2025-04; and in part by the Open Foundation of State key Laboratory of Networking and Switching Technology (Beijing University of Posts and Telecommunications) under Grant SKLNST-2025-1-12. The associate editor coordinating the review of this article and approving it for publication was Dr. Haomiao Yang. (Corresponding author: Zhitao Guan.)

Wenti Yang is with the Peng Cheng Laboratory, Shenzhen 518066, China, and also with the School of Control and Computer Engineering, North China Electric Power University, Beijing 102206, China (e-mail: yangwt@pcl.ac.cn).

Xuan Li and Zhitao Guan are with the School of Control and Computer Engineering, North China Electric Power University, Beijing 102206, China (e-mail: leemac1999@163.com; guan@ncepu.edu.cn).

Meng Li is with the School of Computer Science and Information Engineering, Hefei University of Technology, Hefei 230002, China, also with the Key Laboratory of Knowledge Engineering with Big Data (Hefei University of Technology), Ministry of Education, Hefei 230002, China, and also with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: mengli@hfut.edu.cn).

Zijian Zhang and Liehuang Zhu are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: zhangzijian@bit.edu.cn; liehuangz@bit.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3616623

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

I. INTRODUCTION

THE rapid growth of applications has brought neural networks substantial attention across various domains. From technical perspective, neural networks are known to be computationally intensive, necessitating significant processing capabilities and massive datasets. Thus, cloud services have emerged as major offerings of neural network inference, for users ranging from individuals and small institutions to large companies. However, two crucial concerns arise given the underlying infrastructures of cloud computing [1], [2]:

A. Privacy Preservation

In the scenario where a user requests inference services from a cloud server deployed with neural network models, it is necessary to transmit the data to the cloud server. However, since the cloud server is not fully trusted, sensitive information within the data can be compromised or misused. Moreover, neural network models, which are assets of the service provider and contain sensitive information about a large number of training datasets, cannot be sent directly to the user. As a hedge against this risk, much of the pertinent work [3], [4], [5] has focused on inference based on secure multi-party computation (MPC). This approach partitions the model and data into shares, distributing them among parties for computation. Each party handles only its share, ensuring that the entire set remains inaccessible.

B. Verifiability

In practical scenarios, two issues may result in unfavorable inference results. First, model owners may be inclined to claim an exaggerated accuracy of their models. Second, cloud servers might yield inference results derived from erroneous computational procedures for purposes such as reducing the computational load. Thus, to guarantee authenticity and correctness, it is necessary for users to verify that such inferences are derived by the claimed neural network model. Studies, such as [6], [7], [8], and [9], have illustrated zero-knowledge proof (ZKP) as a potential technique to achieve verifiable neural network inference. However, research in this stream is still in its early stages, as limitations are observed in ZKP-based solutions [10], [11].

Consider the following scenario: AI doctors are introduced to offer healthcare-related support and guidance to patients. Meanwhile, patients may seek services in a manner that

doesn't involve disclosing personal sensitive information and diagnostic results. Furthermore, it's necessary for the patients to have a guarantee of the authenticity of the performance (e.g., accuracy) claimed by the owner of an AI doctor, and the correctness of the diagnostic results. Hence, there is an increasing need to assure both privacy preservation and verifiability to provide a secure and user-friendly service paradigm.

Existing privacy-preserving inference schemes, e.g., MPC-based approaches, ensure the privacy of the models, the data being inferred, and the inference results. In these MPC-based multi-party inference schemes, verifiability becomes even more critical, given the complexity of maintaining the integrity of all participants. Some researchers propose secure multi-party inference schemes with malicious security in the honest-majority setting, capable of tolerating either 1 malicious corruption or $t < n/2$ malicious corruptions [12], [13], [14], where t is the corruption threshold and n is the number of parties. These schemes, however, can only ensure the correctness of the computation when the number of malicious corruptions does not exceed the corruption threshold. A few works provide malicious security with a dishonest majority setting for privacy preserving machine learning, tolerating corruption of up to $n - 1$ parties [15]. Some studies [16], [17] have employed techniques, such as sensitive-samples generation and mix-and-check method, to verify the integrity or correctness of inference results, similar to the concept of sample checking. The reliability of these verifications is probabilistic in nature, and they can only support verification of batch data inference. All the aforementioned schemes, along with some other claims of verifiable and Privacy-Preserving inference schemes [1], [18], [19], [20], [21], fail to ensure the authenticity of the model during the inference process.

In verifiable neural network inference solutions that leverage ZKP, the zero-knowledge property of ZKP ensures the confidentiality of the neural network models. Yet they do not consider the privacy preservation of user's data and the inference results. More specifically, verifiable inference schemes based upon interactive ZKP [8], [9], [22], [23], [24] place additional hurdles in the way to the users due to the necessary interactive proofs. Classic protocols of non-interactive ZKP, also known as zero-knowledge succinct non-interactive argument of knowledge (zk-SNARKs), are also presented as the technical foundation for implementing verifiable inference [6], [7], [25], [26], [27]. While zk-SNARKs incur higher memory requirements, this has been alleviated by their user-friendly nature due to the elimination of interactions.

C. Motivations

As discussed above, none of the existing works have simultaneously addressed the challenges of ensuring the privacy preservation of the neural network models, the data being inferred, and the inference results, as well as verifying the authenticity of neural network models and the correctness of the inference process. Given the existence of well-performing works on MPC-based multi-party inference schemes for privacy preservation, our primary emphasis lies in offering verifiable solutions for existing schemes. We aim to enable verifiable detection of malicious behavior even when all n

parties collude. This is achieved by using zero-knowledge proofs to guarantee computational integrity, ensuring that any deviation from the prescribed computation can be detected. Although full collusion cannot be prevented from producing incorrect outputs, any deviation from the prescribed computation will still be detectable during verification. Moreover, we recognize that not all users (i.e. data owners) have sufficient resources or willingness to participate in the inference process. To eliminate user interaction requirements, we employ zk-SNARKs-specifically the Groth16 protocol-as our underlying ZKP protocol, rather than interactive ZKPs. Groth16 is a verifier-friendly zk-SNARK protocol known for its advantages in proof size (limited to just three points on elliptic curves) and verification time. We mainly focus on scenarios where multi-party computation can be performed by multiple servers, thus relieving users from the burden of multi-party computation [14], [28].

D. Challenges

At first glance, direct integrating MPC and ZKP in the neural network inference process could fulfill the requirements for privacy preservation and verifiability. Prior studies have combined MPC and ZKP to achieve auditable/verifiable MPC [29] or zero-knowledge proofs for secret-shared data [30]. However, achieving privacy preservation and verifiability in neural network inference based on MPC and ZKP presents significant challenges.

The primary challenge lies in the integration of MPC and zk-SNARKs, since MPC-based neural network inference requires collaboration among multiple parties and distributing data and models in a secret sharing manner, whereas the proof generation in zk-SNARKs also needs to be performed by multiple parties. While MPC protocols inherently support operations over any finite field, zk-SNARKs are typically based on elliptic curve groups, leading to substantial computational and communication overhead.

The second challenge involves ensuring the neural network inference process is compatible with both MPC and zk-SNARKs. While existing schemes propose solutions that are compatible with MPC and neural network inference processes, such as scaling and truncation to manage the absence of floating-point number support in MPC, zk-SNARKs introduce further complications. For example, to accommodate the non-linear computation of the linear rectification function (ReLU), a hybrid MPC protocol can be used, which involves applying arithmetic secret sharing for computing linear operation and garbled circuit for computing ReLU. However, zk-SNARKs require converting computations into a quadratic arithmetic programs (QAP) form, which can only be transformed by the product form of the equation. Consequently, directly converting the inequality in ReLU to a QAP form is not feasible.

E. Our Solution

We introduce VSecNN, a verifiable and privacy-preserving neural network inference scheme that addresses the aforementioned challenges. First, we develop a multi-party proof generation approach based on Groth16 [31]. Our approach

TABLE I
COMPARISONS WITH RELATED WORKS

Approach	Privacy			Verifiability		Applicable to NN
	Model	Data	Inference result	Correctness	Authenticity	
HE- or MPC-based privacy-preserving inference [32]–[38]	✓	✓	✓	✗	✗	✓
SafetyNets [22]	✓	✗	✗	✗	✗	✓
vCNN; ZKML; zkLLM; zkCNN; ZEN; psvCNN; ValidCNN [6]–[8], [23]–[27]	✓	✗	✗	✓	✓	✓
Mystique [9]	✓/✗ ^a	✗/✓	✗	✓	✓	✓
MVP [19]; VPMLP [18]	✓	✓	✓	✓	✗	✗
SecureDL [17]; PPVLC [1]	✓	✓	✓	✓	✗	✓
Trident [12]; Falcon [13]; Fantastic four [14]; MD-ML [15]	✓	✓	✓	✓/✗ ^b	✗	✓
pvCNN [39]	✗	✓	✓	✓	✗	✓
vPIN [40]	✗	✓	✓	✓	✓	✓
VSecNN (ours)	✓	✓	✓	✓	✓	✓

^a Mystique provides two solutions: one protects model privacy and the other protects data privacy;

^b These schemes can only ensure correctness when the number of malicious parties is less than the corruption threshold

leverages the conventional finite field-based MPC protocol for inference and the elliptic curve group-based MPC protocol for proof generation. This approach optimizes the integration of MPC and Groth16, enabling efficient multi-party proof generation. Then, we incorporate the multi-party proof generation approach into neural network inference to ensure both privacy-preserving capabilities and verifiability. To better align neural networks with our multi-party proof generation approach, we employ adaptive modifications (e.g., auxiliary parameter injection, polynomial approximation) to the neural network inference process during both: (1) its conversion to QAP representation, and (2) the multi-party inference execution.

F. Contributions

- 1) We introduce VSecNN, a verifiable and privacy-preserving neural network inference scheme. VSecNN not only ensures the privacy of models, input data, and inference results, but also enables the verification of model authenticity and correctness of inference processes in the context of neural network cloud services. It serves as a solution for adding verifiability to certain established MPC-based approaches.
- 2) We develop a multi-party proof generation approach based upon Groth16 with adaptive optimizations to ensure compatibility with the neural network inference process. This includes the adoption of MPC protocol based on elliptic curve groups, and the conversion from neural network to QAP.
- 3) We evaluate our protocol using well-known datasets and neural network models. The experimental results comprehensively validate the performance of our approach.

II. RELATED WORK

Our primary focus is on preserving privacy and ensuring verifiability during neural network inference. Privacy preservation involves protecting the confidentiality of models, inferred data, and inference results. We refer to inference methods that safeguard both the privacy of the model and data privacy as privacy-preserving neural network inference. On the other hand, verifiability pertains to verifying the authenticity of neural network models and the correctness of the inference results. In TABLE I, we provide a comparison between VSecNN and existing state-of-the-art works.

A. Privacy-Preserving Neural Network Inference

Existing approaches are commonly built upon MPC, homomorphic encryption (HE) [32], [33], or a combination of both to ensure privacy preservation. Given the computational efficiency constraints of HE, current research focuses more on MPC and hybrid protocols [41].

Most MPC-based solutions use techniques such as secret sharing, garbled circuits, and homomorphic encryption to design protocols. For instance, the SPDZ (Smart-Pastro-Damgård-Zakarias) [42] protocol, developed by leveraging arithmetic secret sharing and homomorphic encryption, and the ABY (Arithmetic, Boolean, and Yao sharing) [43] protocol, a framework for efficient mixed-protocols MPC, are both extensively used in multi-party secure inference. This stream of research concentrates on improving inference efficiency [3], [34], [35], mitigating the impact of malicious clients [36], [37], and exploring applications for large models [38], [44], among others. However, these approaches do not specifically tackle the challenge of verification. Some works [12], [13], [14], [15] propose maliciously secure schemes to prevent malicious servers from providing incorrect computation results. However, these schemes are only effective when the number of malicious servers is below the corruption threshold, and they cannot guarantee the authenticity of the model.

B. Verifiable Neural Network Inference

Research on verifiable neural network inference using ZKP has grown, falling into two categories: interactive [8], [9], [22], [23], [24] and non-interactive [6], [7], [25], [26], [27]. However, these studies do not consider the privacy preservation of user's data and inference results.

1) *Interactive Verifiable Inference*: SafetyNets [22] is the first to provide verification capabilities for neural networks on untrusted servers. This method adopts the sumcheck protocol, necessitating user interactions with the cloud server to achieve verification. Nevertheless, SafetyNets only guarantees the correctness of the computation process, without addressing the verification of whether the designated model is applied by the service provider. zkCNN [23] follows a similar approach by leveraging the sumcheck protocol, to validate the prediction results of the neural network. zkLLM [8] adopts the sumcheck protocol tailored for verifying large

language model inferences while preserving model privacy. It employs specialized parallelized lookup arguments and a dedicated ZKP for the attention mechanism, achieving efficient verifiable computations. Other relevant studies are Mystique [9] and [24], which employ vector oblivious linear evaluation (VOLE) for achieving verifiable neural network inference.

2) *Non-Interactive Verifiable Inference*: zk-SNARKs serve as the primary technique for achieving non-interactive verifiable inference, such as vCNN [6] and ZEN [25]. These solutions focus on how to improve the efficiency of zk-SNARKs in neural network inference. ZKML [7] is a framework for generating zk-SNARKs for realistic machine learning models. It optimizes TensorFlow model translation into zk-SNARK circuits to enhance proving and verification efficiency. Both psvCNN [26] and ValidCNN [27] focus on verifying the integrity of CNN predictions using zk-SNARKs, leveraging Freivald's algorithm to reduce proof time and storage costs.

C. Verifiable and Privacy-Preserving Inference

An increasing emphasis is also placed on simultaneously addressing privacy preservation and verifiability. Liu et al. [1] propose PPVLC, which uses a secret sharing protocol to protect privacy and a blinding technique to verify the correctness of the inference. MVP [19] enables verifiable and privacy-preserving prediction for support vector machines (SVMs) using the BGN (Boneh-Goh-Nissim) homomorphic cryptosystem and a signature algorithm. However, MVP does not offer a solution for neural network inference. Similarly, VPMLP [18] is limited to supporting predictions from linear regression models. Xu et al. [17] employ an HE scheme and a sensitive-samples generation method to achieve secure and verifiable inference, but the validity of their verification process is probabilistic in nature. Furthermore, none of the above solutions have considered the authenticity of the neural network models. pvCNN [39] integrates HE and zk-SNARKs to ensure both privacy and verifiability in CNN testing. The model is split into PriorNet, kept private by the developer, and LaterNet, delegated to a server with weaker privacy protection. However, their approach only ensures verifiability on the server side. vPIN [40] integrates additive HE (AHE) and commit-and-prove SNARK to protect client data privacy while ensuring the verifiability. To overcome the limitation of AHE, which does not support ciphertext multiplication, vPIN delegates activation function computation to the client, who decrypts, processes, and re-encrypts intermediate results. However, this approach introduces security risks, as clients may infer model details from intermediate data, and requires significant client-side computation, reducing practicality.

III. PRELIMINARIES

A. Bilinear Maps

Let $\mathbb{BG} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p, g, h)$ denote a bilinear group, where $\mathbb{G}_1$, $\mathbb{G}_2$ and $\mathbb{G}_T$ are elliptic curve groups of prime order p , g and h are generators of $\mathbb{G}_1$ and $\mathbb{G}_2$ respectively. The bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ satisfies $e(au, bv) = ab \cdot e(u, v)$ for all $a, b \in \mathbb{Z}_p$, $u \in \mathbb{G}_1$, $v \in \mathbb{G}_2$. For convenience, We denote

$a \cdot g$ as $[a]_1$, $b \cdot h$ as $[b]_2$, and $e(ag, bh)$ as $[a]_1 \cdot [b]_2$ in the subsequent sections.

B. Secure Multi-Party Computation

MPC includes a series of cryptographic protocols that enable N mutually distrustful parties to jointly compute a function f over private inputs $x_i (i = 1, \dots, M)$, which may originate from separate data owners. We denote such a protocol as Π , in which N parties jointly compute $y = f(x_1, x_2, \dots, x_M)$. The protocol Π is said to be secure if no party gains any knowledge beyond the final output y .

An MPC protocol based on arithmetic secret sharing is commonly divided into the online phase and the offline phase. Therefore, a portion of resource-intensive operations can be shifted to the offline phase. During the offline phase, Beaver multiplication triples are generated through methods such as homomorphic encryption. Subsequently during the online phase, arithmetic operations, including addition and multiplication, can be performed cooperatively among the N participating parties. Consider a case involving M confidential input $x_i (i = 1, \dots, M)$ over a finite field F_p . Each confidential input x_i is distributed among N parties as N shares $(\langle x_i \rangle_1, \dots, \langle x_i \rangle_N)$, where $x_i = \langle x_i \rangle_1 + \dots + \langle x_i \rangle_N$. Each party P_i then interactively computes $\langle y \rangle_i = f(\langle x_1 \rangle_i, \dots, \langle x_M \rangle_i)$. This computation may include the addition of two shared values, the multiplication of a constant and a shared value, and the multiplication of two shared values.

For the addition operation $y = \sum_{j=1}^M x_j$, each party P_i holding $\langle x_j \rangle_i$, simply computes $\langle y \rangle_i = \sum_{j=1}^M \langle x_j \rangle_i$. In the case of $y = ax$, where a is a constant and x is a confidential input, each party P_i , possessing a and $\langle x \rangle_i$, only needs to compute $\langle y \rangle_i = a \langle x \rangle_i$. These two operations do not necessitate any interaction among N parties, nor additional parameters. Finally, y can be recovered by summing up $y = \sum_{i=1}^N \langle y \rangle_i$.

However, for the operation $y = \prod_{j=1}^M x_j$, the N parties cannot simply derive $\langle y \rangle_i$ by calculating $\langle y \rangle_i = \prod_{j=1}^M \langle x_j \rangle_i$. The multiplication of two shared values mandates interactions between parties, and each multiplication operation consumes one Beaver multiplication triple, which is generated during the offline phase. Given that our primary focus is on zero knowledge proofs for the neural network inference phase, we do not delve into the specifics of the algorithm for secure multi-party computation here. Please refer to References [28], [29] for more information.

C. Groth16

Groth16 is a zk-SNARK variant known for its efficiency in proof size and verification time. Despite requiring a trusted setup, Groth16 remains one of the most widely used zk-SNARK schemes in practice, especially in scenarios for rapid verification. Compared with transparent setup zk-SNARKs, Groth16 offers succinct proofs (only 3 group elements) and constant-time verification, making it particularly suitable for scenarios where low verification overhead is crucial. Groth16 allows a prover to present a proof π for a relation R , demonstrating the existence of a statement x and a witness w that fulfill $R(x, w) = 1$. The statement x represents the public claim

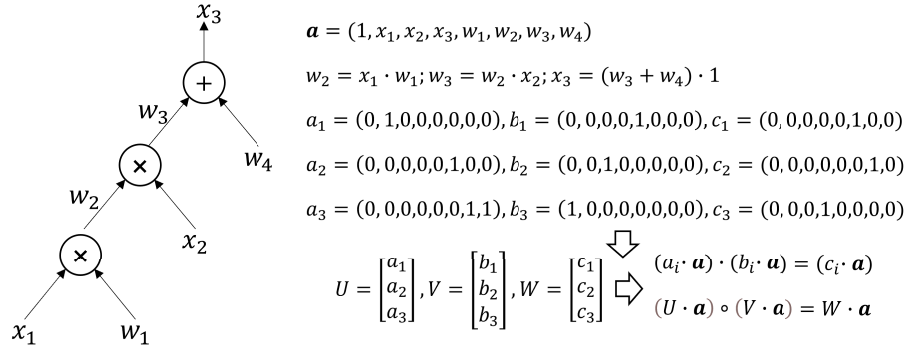


Fig. 1. The conversion of an arithmetic circuit into RICS.

being proven while the witness w is the private information known only to the prover, used to verify the truth of x . The verifier can check the proof in $O(|R|)$ time without learning anything about w . The relation R is expressed as a quadratic arithmetic program (QAP), derived from a rank 1 constraint system (RICS), which encodes variable dependencies in an arithmetic circuit. Below we illustrate the concepts of RICS and QAP by demonstrating how an arithmetic circuit is first transformed into RICS and then subsequently converted to QAP.

As shown in Fig. 1, consider an arithmetic circuit C with statement wires $\mathbf{x} = (x_1, x_2, x_3)$ and witness wires $\mathbf{w} = (w_1, w_2, w_3, w_4)$. The prover's goal is to show that they possess a witness $\mathbf{w}$ that satisfies C . Let $\mathbf{a} = (a_0, a_1, \dots, a_n) = (1, \mathbf{x}, \mathbf{w})$, U, V, W are three $m \times (n+1)$ matrices, the RICS is expressed as:

$$(U \cdot \mathbf{a}) \circ (V \cdot \mathbf{a}) = W \cdot \mathbf{a} \quad (1)$$

where m is the number of constraints, and n equals $|\mathbf{a}| - 1$.

We define a basis $(r_1, r_2, \dots, r_m)$, and for each element of the matrices U, V, W , we set $U_{i,j} = u_j(r_i)$, $V_{i,j} = v_j(r_i)$, $W_{i,j} = w_j(r_i)$. For this basis, the following m equations hold:

$$\sum_{j=0}^n a_j u_j(r_i) \cdot \sum_{j=0}^n a_j v_j(r_i) = \sum_{j=0}^n a_j w_j(r_i) \quad (2)$$

Consequently, each column j of the matrices can be converted to QAP polynomials $u_j(X)$, $v_j(X)$, $w_j(X)$ with degree $m-1$, through the process of Lagrange interpolation polynomial. We introduce a zero polynomial $t(X) = \prod_{i=1}^m (X - r_i)$ with degree m , the following equation holds:

$$\sum_{j=0}^n a_j u_j(X) \cdot \sum_{j=0}^n a_j v_j(X) - \sum_{j=0}^n a_j w_j(X) = h(X)t(X) \quad (3)$$

where $h(X)$ is the quotient polynomial. Finally, a QAP relation R for all $(x, w) \in R$ is represented as:

$$R = (\text{aux}, \{u_j(X), v_j(X), w_j(X)\}_{j=0}^n, t(X)) \quad (4)$$

where aux is auxiliary parameters.

The Groth16 protocol is implemented by the following algorithm:

- $ZK_Setup(1^\lambda, R) \rightarrow CRS$. Given the security parameter λ and the relation R , $ZK_Setup()$ generates a common reference string (CRS) linked to the relation R .

- $Prove(R, CRS, \mathbf{x}, \mathbf{w}) \rightarrow \pi$. Given the relation R , the CRS, the statement $\mathbf{x}$, and the witness $\mathbf{w}$ as inputs, $Prove()$ generates the proof π .
- $Verify(R, CRS, \mathbf{x}, \pi) \rightarrow \{0, 1\}$. Given the relation R , the CRS, the statement $\mathbf{x}$, and the proof π as inputs, $Verify()$ outputs 1 if the verification is successful; otherwise, it outputs 0.

D. KZG Polynomial Commitments

KZG polynomial commitments [45] are key cryptographic primitives in zk-SNARKs. By integrating them with a commitment protocol, zk-SNARKs ensure the prover commits to designated values before proof generation, preventing tampering during verification. KZG commitments allow provers to efficiently commit to vectors, making them well-suited for neural networks with numerous parameters. Moreover, while KZG requires a structured reference string (SRS), it provides constant-size commitments, which is crucial for reducing proof size in our setting. The protocol is defined as follows.

- $KZG_Setup(1^\lambda) \rightarrow SRS$. Given the security parameter λ as input, $KZG_Setup()$ generates the SRS. The SRS for the KZG commitment is a special type of CRS. It is universal and remains independent of the committed polynomial.
- $Commit(SRS, W) \rightarrow cm_W$. Given the SRS and the data W that need to be committed as inputs, $Commit()$ generates the commitment cm_W .
- $Com_Open(SRS, cm_W, W) \rightarrow \{0, 1\}$. Given the SRS, the commitment cm_W , and the data W as inputs, $Com_Open()$ is used to reveal the commitment.

In zk-SNARKs, it is generally unnecessary to invoke $Com_Open()$ to reveal commitments. Hence, the KZG commitment protocol preserves the zero-knowledge property. Specifically, commitments serve to protect the prover's confidential data and are included in the proof submitted to the verifier. Since commitments and other public values are available for verification, the verifier cannot directly access the confidential data. In summary, due to the computationally binding and statistically hiding properties of the commitment protocol, the prover is prevented from modifying the committed data, while the verifier cannot extract any sensitive information pertaining to the committed data [46]. This is particularly crucial for secure MPC, ensuring both computational

correctness and the integrity, consistency, and privacy of the data.

IV. OVERVIEW

A. System Model

There are five major entities in VSecNN, including the trusted third party, model tester, model owner, data owner, and N servers.

- 1) The **Trusted Third Party** is responsible for the initial setup of the system. It generates both the CRS for Groth16 and the SRS for KZG commitments. The party ensures that the secret keys used during the generation of the CRS and SRS are kept confidential and are securely destroyed after the initialization phase, thus establishing a solid cryptographic foundation for the entire protocol.
- 2) The **Model Tester** plays a critical role in verifying model authenticity before deployment. The model tester is defined as any party, such as an external auditor, a regulatory agency, or a prospective client, who seeks to verify the model's claimed performance using their own benchmark dataset before deployment. Prior to the model being put into production, the model tester provides a test dataset to the model owner. The model tester then checks the inference results and the corresponding zero-knowledge proof produced by the model owner, ensuring that the model committed by the model owner is indeed the one used during inference and that the claimed accuracy is validated.
- 3) The **Model Owner** first commits to the model parameters to ensure that the evaluated model is used in secure inference. Next, it generates the QAP relation and tests the model with the public dataset from the model tester, producing a zero-knowledge proof that validates the fixed parameters and claimed accuracy before deployment. For secure inference, the model owner splits the model parameters into N secret shares and distributes them to N servers.
- 4) The **Data Owner**, who requires inference, partitions its secret input data into N shares via secret sharing and sends these shares to the N servers. After the inference is completed, the data owner receives the result along with the associated zero-knowledge proof and verifies the correctness of the inference computation to ensure that the process was executed as intended.
- 5) The N **Servers** collaboratively perform secure inference by processing secret-shared model parameters and input data. They jointly compute the inference result and generate a zero-knowledge proof attesting to the computation's correctness, thus preserving data privacy and ensuring result integrity.

B. VSecNN Workflow

In order to achieve both verifiability and privacy preservation, we incorporate Groth16 into multi-party neural network secure inference to achieve VSecNN. The verifiability of neural network inference involves two key aspects. The first aspect is to commit the model parameters, ensuring the

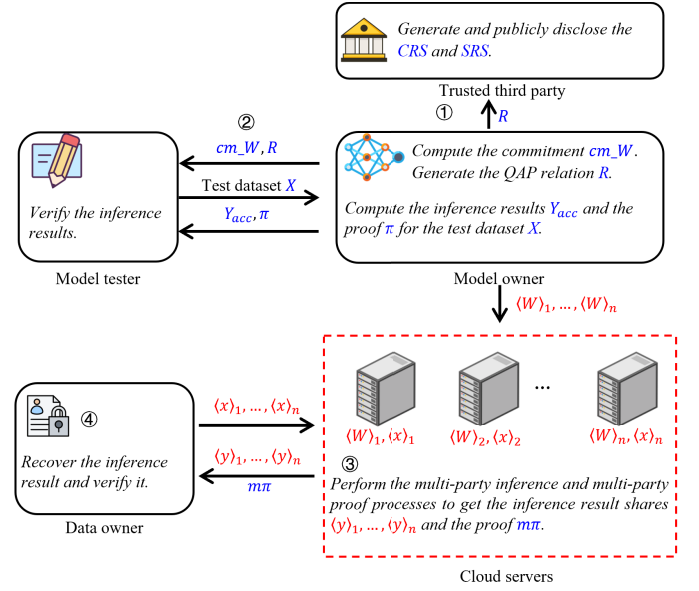


Fig. 2. Overview of VSecNN workflow. Note that the values highlighted in red represent the secret values, while those in blue represent the publicly available values.

authenticity of the model. The second aspect is to confirm the correctness of the forward propagation algorithm, ensuring that each party carries out the inference in compliance with the agreed protocol. In Fig. 2, we present the overview of VSecNN. The model owner begins by creating a commitment cm_W based on the parameterized model W to ensure that W is designated for subsequent inference. Then any model tester can interact with the model owner to validate the accuracy of the model. Subsequently, both the model W and the data x are split into N secret shares and transmitted to N servers. These servers then collaborate to generate secret shares and a proof of the inference result y , which then dispatched to the data owner/verifier. Hence, the data owner/verifier is able to regenerate y using the secret shares and verify whether y matches the correct inference results given W and x . Throughout this process, sensitive information of user's data, model parameters, and inference results remain undisclosed.

The workflow of VSecNN primarily includes initialization, proof of accuracy, multi-party inference, multi-party proof, and verification. We elaborate each step as follows.

1) **Initialization**: This step involves the generation of the QAP relation R for the neural network model and the trusted setup of Groth16 and KZG commitment.

2) **Proof of Accuracy**: Before providing model inference to users, the model owner generates a public commitment cm_W for the neural network model W . Any user (model tester) can submit a test dataset to the model owner and receive the inference results along with a zero-knowledge proof of the committed model. Consequently, the model testers are capable of verifying the proof, in order to confirm that the committed model has achieved the claimed accuracy. This step ensures the authenticity of the model.

3) **Multi-Party Inference**: The model owner splits the model parameters W into N shares and distributes them across N

servers. When a data owner conducts an inference task, they also partition their data d into N shares and distribute them to the N servers. These N servers then collaborate to carry out the inference on the neural network model using the data, ultimately delivering the inference result shares back to the user. More importantly, this process is designed in a way such that no sensitive information regarding the model or the data is revealed to the N servers.

4) *Multi-Party Proof*: During the neural network inference process, the N servers are also required to collaboratively generate a zero-knowledge proof, which is then sent to the user. This step ensures the verifiability of the inference result.

5) *Verification*: Upon receiving the inference result shares, the user can verify it using the associated zero-knowledge proof. A failed verification indicates either at least one party has not adhered to the agreed computation protocol, or incorrect model parameters have been applied during the inference.

C. Threat and Security Model

We consider the active adversary model for the N servers, with the capability to actively manipulate, interfere with, or modify data. Unlike the passive adversary model assumed in most MPC-based solutions, where the adversary is limited to passive eavesdropping and information gathering, the active adversary model implies a more powerful attacker. VSecNN demonstrates effective resilience against collusion attacks by $N-1$ servers. In other words, even if $N-1$ servers collude, the private information of the data cannot be recovered. On the other hand, we assume the passive adversary model regarding the model owner, model tester, and data owner. They might attempt to extract privacy information from neural network models or the data inferred, but they do not actively engage in modifying, disrupting, or manipulating the transmitted data [47].

The security of VSecNN is analyzed through three aspects:

- 1) *Proof of accuracy*: VSecNN is built on Groth16, wherein the confidentiality of the model parameters, alongside the completeness and soundness of the accuracy proof are assured by the secure properties of Groth16, including completeness, soundness, and zero-knowledge [31].
- 2) *Multi-party inference*: Model parameters and data are partitioned into N shares using secret sharing techniques, thereby inheriting the security features of secret sharing.
- 3) *Multi-party proof*: Let Γ be the protocol of the multi-party proof, Γ is considered to be secure against N malicious servers, if it satisfies the properties of completeness, knowledge soundness, and N -zero-knowledge as defined below.

1) *Definition 1 (Completeness)*: If N honest servers each possesses a secret share of a statement and a witness, they can collaboratively generate a proof. The verifier has a negligible probability $\text{negl}(\lambda)$ of rejecting it (outputting 0), where λ is the security parameter.

2) *Definition 2 (Knowledge Soundness)*: For any t , ($t \leq N$) computationally bounded adversaries $\mathcal{A}_i$ ($i = 1, \dots, t$) who are unaware of the witness, there exists a computationally bounded extractor $\mathcal{E}$ with complete access to the states of t

adversaries. This extractor $\mathcal{E}$ can derive a witness w whenever the t adversaries and $N-t$ honest servers jointly generate a valid proof π , such that the probabilities of $(x, w) \notin R$ and the proof π convincing the verifier are negligible.

3) *Definition 3 (N-Zero-Knowledge)*: For a computationally bounded adversary $\mathcal{A}$, it has a negligible probability $\text{negl}(\lambda)$ of learning any information about the secret shares of the witness from the N proof shares generated by N servers.

V. PROPOSED SCHEME

In this section, we provide a detailed description of VSecNN. Consider a CNN model with parameters W (weights and biases). The inference process, also known as forward propagation, can be represented as the solution of the function $f(d, W)$, where d represents the input data undergoing inference. First, we describe the methodology for converting the inference process of W into a QAP relation, along with the trusted setup for this relation. Subsequently, we illustrate how the model owner can validate the claimed accuracy of the model. This is followed by outlining the procedures for multi-party inference and multi-party proof. Lastly, we show how the data owner can verify the inference result.

A. Neural Network to QAP

In order to improve the compactness and efficiency of the proof process, the inference process is first converted into a QAP form. This begins with representing $f(d, W)$ using an arithmetic circuit over a finite field F_p with addition gates and multiplication gates. Subsequently, these gates are mapped onto R1CS constraints. The entire R1CS constraint system is eventually converted into a QAP form. In this setup, we designate a portion of the input wires (i.e., the input data) and the output wires (i.e., the inference result) as the statement, while the remaining wires (i.e., the weights, biases, and intermediate values) are defined as the witness. This process results in a QAP relation R' , which consists of the statement and the witness fulfilling the equation (3) specified in Section III-C. A model owner can then generate a proof based on R' , to convince a data owner that they possess a neural network model W , from which the result y is derived through model inference on data d .

However, a proof based on R' can only guarantee that the inference process satisfies the circuit specifications. It implies that any neural network conforming to the circuit can be used for inference. To ensure the model owner consistently uses the committed model, it is necessary to embed the committed computation process into the circuit. Hence, the statement should also incorporate the commitment value cm_W of the model parameters W , in addition to the input data d and inference result y .

As a result, a QAP relation R is derived as

$$R_{f_cm} = (\mathbb{B}\mathbb{G}, l, \{u_i(X), v_i(X), w_i(X)\}_{i=0}^n, t(X)) \quad (5)$$

characterized by:

- The QAP polynomials $\{u_i(X), v_i(X), w_i(X)\}_{i=0}^n$ of degree $m-1$, are derived from a circuit, which includes both the

inference process of the neural network model and the commitment for the model parameters W .

- The zero polynomial $t(X)$ of degree m , and the quotient polynomial $h(X)$.
- The statement $x = (a_1, \dots, a_l)$, including the input data d , the commitment values cm_W , and the inference result y .
- The witness $w = (a_{l+1}, \dots, a_n)$, consisting of the model parameters W such as the weights, biases, as well as the intermediate values generated from the inference.
- $(x, w) \in R_{f_cm}$ if it satisfies: $\sum_{i=0}^n a_i u_i(X) \cdot \sum_{i=0}^n a_i v_i(X) = \sum_{i=0}^n a_i w_i(X) + h(X)t(X)$, where $a_0 = 1$.

Note that m is the number of constraints from both the forward propagation algorithm and the commitment algorithm. Optimizing the number of constraints, which significantly influences the proof generation efficiency, can be achieved through more sophisticated constraint development methods.

Another challenge arises is some operations in the CNN forward propagation algorithm cannot be directly converted into the QAP form. Specifically, linear operations in the convolutional, fully connected, and average pooling layers are straightforward. However, non-linear operations in activation functions, such as exponential and comparison operations, cannot be efficiently represented by multiplication and addition gates. As a result, it is impossible to directly convert them into the QAP form. There are two ways to address this challenge, the first method is to utilize polynomial approximation methods, which allow non-linear operations to be represented as polynomials. These polynomials can then be converted into the QAP form directly. The second method is specifically designed for ReLU. We accomplish the QAP construction of ReLU by adding an auxiliary parameter. Specifically, during the inference process, apart from outputting the result of ReLU, an additional Boolean value tmp is also produced to represent the comparison between the input data and 0. For example, if x is the input of ReLU, and $x > 0$, then it outputs x and $tmp = 1$. Conversely, if $x \leq 0$, it outputs 0 and $tmp = 0$. Therefore, the comparison operation in ReLU can be represented as a multiplication gate $x' = tmp \cdot x$, which can be directly converted to the QAP form.

B. Trusted Setup

This step includes the trusted setup for both the Groth16 and the KZG commitment where a shared set of values known as the CRS and SRS is derived. The specifics are as follows:

$ZK_Setup(1^\lambda, R) \rightarrow \{pk, sk\}$: The trusted third party randomly generates secret elements $\alpha, \beta, \gamma, \delta, \varepsilon \leftarrow \mathbb{Z}_p^*$, and computes:

$$pk_1 = \left\{ \begin{array}{l} \alpha, \beta, \delta, \{\varepsilon^i\}_{i=0}^{m-1} \\ \{\gamma^{-1}(\beta u_i(\varepsilon) + \alpha v_i(\varepsilon) + w_i(\varepsilon))\}_{i=0}^l \\ \{\delta^{-1}(\beta u_i(\varepsilon) + \alpha v_i(\varepsilon) + w_i(\varepsilon))\}_{i=l+1}^n \\ \{\delta^{-1}(\varepsilon^i t(\varepsilon))\}_{i=0}^{m-2} \end{array} \right\} \quad (6)$$

$$pk_2 = (\beta, \gamma, \delta, \{\varepsilon^i\}_{i=0}^{m-1}) \quad (7)$$

$$sk = (\alpha, \beta, \gamma, \delta, \varepsilon) \quad (8)$$

The CRS, represented as $pk = ([pk_1]_1, [pk_2]_2)$, involves computing $[x]_1 = x \cdot g$ for each element x in pk_1 as $[pk_1]_1$, and computing $[y]_2 = y \cdot h$ for each element y in pk_2 as $[pk_2]_2$.

$KZG_Setup(1^\lambda) \rightarrow PK$: The trusted third party randomly generates a secret element $\theta \leftarrow \mathbb{Z}_p^*$, and then computes: $PK = ([\theta^k]_1, [\theta]_2)$ as the SRS, where k is the degree of a polynomial that can be committed to using this SRS.

C. Proof of Accuracy

Prior to offering inference services of a neural network model to end users, the model owner is required to provide a publicly verifiable proof of the model's claimed accuracy, to ensure the authenticity of the model. Specifically, before deployment, the model owner must first generate a public commitment to the model parameters. Then, upon receiving a test dataset from the model tester, the model owner computes the inference results on this dataset and produces a zero-knowledge proof attesting to the correctness of these results with respect to the committed model. This enables the model tester to verify the authenticity of the claimed accuracy without accessing the model itself. Moreover, the public commitment is reused in subsequent inference services, ensuring that the model used in future computations remains identical to the one that was initially validated. The specific process is as follows:

1) Transforming Model Parameters W Into Polynomials:

The model owner starts with a neural network model W , claimed to achieve accuracy e . To commit to the model parameters using KZG polynomial commitments, the model owner first transforms the neural network parameters W into a structured format suitable for polynomial representation. Specifically, the model parameters in W are sequentially grouped into vectors of $k + 1$ elements, denoted as $v = (v_1, \dots, v_{k+1})$, where k is the polynomial predetermined during parameter initialization. If the number of parameters is not a multiple of $k + 1$, appropriate zero-padding is applied to complete the final vector. Each vector v is then mapped to a polynomial g of degree k using Lagrange interpolation, such that the polynomial satisfies $g(i) = v_i$ for $i = 1, 2, \dots, k + 1$. This means that the values of the polynomial at those evaluation points recover the original vector elements. The set of all such interpolated polynomials derived from W is denoted as $g_W = \{g_1, g_2, \dots\}$ where each g_i corresponds to one vector segment of the model parameters. Once the polynomials g_W are constructed, the model owner applies KZG commitments to them.

2) Commitment Generation: After transforming the model parameters W into a set of polynomials $g_W = \{g_1(X), g_2(X), \dots\}$, the model owner proceeds to generate commitments for each polynomial using the KZG commitment scheme. Given a polynomial $g = \sum_{i=0}^k c_i X^i$, the corresponding KZG commitment is computed as $cm = \sum_{i=0}^k c_i [\theta^i]_1$, where $[\theta^i]_1$ denotes elements of the SRS in group $\mathbb{G}_1$ and c_i are the polynomial coefficients. We define the commitment of all the polynomials transformed by the model parameters W as cm_W . The commitments collectively serve as a public cryptographic representation of the model's parameters. Once computed, cm_W is published

to all users, ensuring that the committed model remains fixed and verifiable throughout both testing and deployment phases.

3) *Accuracy Test*: To verify that the committed model indeed achieves the claimed accuracy e , a model tester can initiate an accuracy testing process by submitting a test dataset D that contains no sensitive information to the model owner. Upon receiving the dataset D , the model owner performs inference using the committed model W , obtaining the inference result $Y' = f(D, W)$. To convince the model tester that the inference was carried out correctly with respect to the committed model (i.e., the exact model bound by the commitment cm_W), the model owner also generates a zero-knowledge proof π using the proof generation algorithm $\pi = \text{Prove}(R_{f_cm}, pk, D, W)$. Finally, the model owner returns both Y' and π to the model tester for verification.

4) *Verification*: After receiving the inference result Y' and the associated proof π , the model tester can verify the inference result by executing $\text{Verify}(R_{f_cm}, pk, D, Y', \pi) \rightarrow \{0, 1\}$ to confirm that Y' is resulting from the committed model W on dataset D . Subsequently, the user compares Y' against the actual label Y of dataset D to determine if the committed model has achieved the claimed accuracy e on test dataset D .

D. Multi-Party Inference

In the practices of cloud-based neural network inference, MPC-based approaches are typically used to ensure the privacy of both the model and data. The MPC-based secure inference is developed based on the protocol Π , such that:

$$\Pi(f(\{\langle d \rangle_j, \langle W \rangle_j\}_{j=1}^N)) \rightarrow \{\langle y \rangle_j\}_{j=1}^N \quad (9)$$

First, the parameters of W and the test data d are confidentially shared among N servers $P_j (j = 1, \dots, N)$. Each server P_j holds the input data shares $\langle d \rangle_j$ and the model parameter shares $\langle W \rangle_j$. Next, the forward propagation algorithm f is carried out under the protocol Π by these servers, including linear operations such as convolution and dot product, and non-linear operations such as ReLU. The inference result shares $\{\langle y \rangle_1, \dots, \langle y \rangle_N\}$ are sent to the data owner.

As discussed in the Section III-B, arithmetic secret sharing supports only linear operations, resulting in incomplete support for performing the entire forward propagation algorithm. Studies [12,20] have focused on hybrid MPC protocols between linear (e.g., SPDZ) and non-linear (e.g., garbled circuit, Boolean sharing) operation-friendly protocol to address this issue. Our scheme provides verifiability for such secure inference protocols but may require minor adjustments when constructing QAPs for non-linear operations. For example, if the polynomial approximation method is used for activation functions in the neural network to QAP phase, it is crucial that the same polynomial approximation method is employed for the activation function in the multi-party inference phase as well. This ensures successful verification, as the computations are approximate and consistency in methodology is necessary. If the activation function is ReLU, then only the additional Boolean value tmp needs to be output during the inference phase. Given the accuracy of the computations, no further modifications are required to ensure successful verification.

E. Multi-Party Proof

In addition to calculating $y = f(d, W)$, the N servers also need to collaborate to generate a zero-knowledge proof, requiring a multi-party proof generation approach to verify the correctness of the computation. In this process, all non-public computation parameters—including the statement, witness, and other parameters necessary for proof generation—are secret-shared among the N servers. We introduce each of these components in detail below.

- The statement is represented as $\{a_i\}_{i=1}^l$, and the corresponding secret shares are denoted by $\{\langle a_i \rangle_1, \dots, \langle a_i \rangle_N\}_{i=1}^l$. This includes the input data shares provided by the data owner, the shares of the model commitment cm_W , and the inference result shares generated during the multi-party inference phase.
- The witness is denoted as $\{a_i\}_{i=l+1}^n$, with corresponding shares $\{\langle a_i \rangle_1, \dots, \langle a_i \rangle_N\}_{i=l+1}^n$. These shares consist of the shares of the model parameters W from the model owner, and the intermediate values computed by N servers during the multi-party inference phase.
- The quotient polynomial $h(\epsilon)$, as introduced in Section III-C, need to be computed by the model owner first and then secret-shared among the servers in the form of $\langle h(\epsilon) \rangle_1, \dots, \langle h(\epsilon) \rangle_N$.
- The QAP relation R_{f_cm} , which is associated with the forward propagation function f and the KZG commitment, and the CRS, represented as $pk = ([pk_1]_1, [pk_2]_2)$, as introduced in Section V-B, are public to all N servers.

A proof for an MPC protocol $\Pi(f)$ is generated through the collaboration of these N servers, satisfying $R_{f_cm}(\{\langle a_i \rangle_1, \dots, \langle a_i \rangle_N\}_{i=1}^n) = 1$. The process of multi-party proof, denoted as $M_Prove(R_{f_cm}, pk, \{\langle a_i \rangle_1, \dots, \langle a_i \rangle_N\}_{i=1}^n) \rightarrow m\pi$, unfolds as follows:

- 1) Each server P_j holds the statement shares $\{\langle a_i \rangle_j\}_{i=1}^l$, the witness shares $\{\langle a_i \rangle_j\}_{i=l+1}^n$, $\langle h(\epsilon) \rangle_j$, the relation R_{f_cm} , and the CRS.
- 2) Each server P_j randomly generates two elements $\langle r \rangle_j$, $\langle s \rangle_j$, where $r = \sum_{j=1}^N \langle r \rangle_j$, $s = \sum_{j=1}^N \langle s \rangle_j$.
- 3) P_1 computes $[\langle A \rangle_1]_1, [\langle B \rangle_1]_2, [\langle C \rangle_1]_1$, where

$$\langle A \rangle_1 = \alpha + u_0(\epsilon) + \sum_{i=1}^n \langle a_i \rangle_1 u_i(\epsilon) + \langle r \rangle_1 \delta \quad (10)$$

$$\langle B \rangle_1 = \beta + v_0(\epsilon) + \sum_{i=1}^n \langle a_i \rangle_1 v_i(\epsilon) + \langle s \rangle_1 \delta \quad (11)$$

$$\langle C \rangle_1 = \frac{\sum_{i=l+1}^n \langle a_i \rangle_1 (\beta u_i(\epsilon) + \alpha v_i(\epsilon) + w_i(\epsilon)) + \langle h(\epsilon) \rangle_1 t(\epsilon)}{\delta} + \langle A \rangle_1 \langle s \rangle_1 + \langle B \rangle_1 \langle r \rangle_1 - \langle r \rangle_1 \langle s \rangle_1 \delta \quad (12)$$

- 4) The other server P_j computes $[\langle A \rangle_j]_1, [\langle B \rangle_j]_2, [\langle C \rangle_j]_1$, where

$$\langle A \rangle_j = \sum_{i=1}^n \langle a_i \rangle_j u_i(\epsilon) + \langle r \rangle_j \delta \quad (13)$$

$$\langle B \rangle_j = \sum_{i=1}^n \langle a_i \rangle_j v_i(\epsilon) + \langle s \rangle_j \delta \quad (14)$$

$$\langle C \rangle_j = \frac{\sum_{i=l+1}^n \langle a_i \rangle_j (\beta u_i(\varepsilon) + \alpha v_i(\varepsilon) + w_i(\varepsilon)) + \langle h(\varepsilon) \rangle_j t(\varepsilon)}{\delta} + \langle A \rangle_j \langle s \rangle_j + \langle B \rangle_j \langle r \rangle_j - \langle r \rangle_j \langle s \rangle_j \delta \quad (15)$$

- 5) The server P_j sends $[\langle A \rangle_j]_1, [\langle B \rangle_j]_2, [\langle C \rangle_j]_1$ to P_1 , which then computes:

$$[A]_1 = \sum_{j=1}^N [\langle A \rangle_j]_1, [B]_2 = \sum_{j=1}^N [\langle B \rangle_j]_2, [C]_1 = \sum_{j=1}^N [\langle C \rangle_j]_1 \quad (16)$$

Finally, it computes $m\pi = ([A]_1, [B]_2, [C]_1)$, and sends the proof $m\pi$ to the data owner. This results in a proof that necessitates only three group elements.

We now introduce optimization approach for the above process. A naïve approach to develop multi-party proof is by directly running the proof generation function of Groth16 under the MPC protocol among N servers. In general, an MPC protocol supports secure arithmetic circuits computations over any finite field F_p , whereas zk-SNARKs are typically based on the elliptic curve groups. The computations of $[A]_1, [B]_2, [C]_1$ require scalar multiplications of the elliptic curve. However, performing an elliptic curve addition under a finite field-based MPC protocol results in approximately six field additions and three field multiplications, as a consequence of mapping elliptic curve group operations onto finite field operations. This implies that even a simple elliptic curve addition demands three multi-party field multiplications and interactions. Therefore, the scalar multiplication in an elliptic curve may lead to thousands of field operations. For instance, the three scalar multiplications $[A]_1, [B]_2, [C]_1$ require a total of $|A + B + C|$ elliptic curve additions. In this context, A, B, C represent large integers in F_p , and their magnitudes increase linearly with the computational complexity of the relation R . As a result, the cooperative generation of a proof $m\pi$ for a neural network by multiple servers under the MPC protocol may incur millions of finite field operations. In addition, an even worse scenario may occur, as a field multiplication under the MPC protocol inherently slows down by a thousand times compared to direct execution. All of these factors render the naïve approach infeasible due to the potential substantial overhead.

We therefore provide a solution by substituting the MPC protocol over a finite field with the one over elliptic curve groups as necessary. Specifically, we use the finite field based MPC protocol to achieve secure multi-party inference, and the elliptic curve groups based MPC protocol to accomplish multi-party proof. In particular, the MPC operations over a finite field are extended to $Add_F, Mul_F, Add_G, Mul_GP, Mul_GS$ as follows:

$$Add_F(\langle a \rangle_i^F, \langle b \rangle_i^F) \rightarrow \langle a + b \rangle_i^F \quad (17)$$

$$Mul_F(\langle a \rangle_i^F, \langle b \rangle_i^F) \rightarrow \langle a \cdot b \rangle_i^F \quad (18)$$

$$Add_G(\langle Q \rangle_i^G, \langle R \rangle_i^G) \rightarrow \langle Q + R \rangle_i^G \quad (19)$$

$$Mul_GP(\langle a \rangle_i^F, R_i^G) \rightarrow \langle a \cdot R \rangle_i^G \quad (20)$$

$$Mul_GS(\langle a \rangle_i^F, \langle R \rangle_i^G) \rightarrow \langle a \cdot R \rangle_i^G \quad (21)$$

where $\langle \cdot \rangle_i^F$ and $\langle \cdot \rangle_i^G$ denote the shared values of an element in a finite field F_p and an element in an elliptic curve group

$\mathbb{G}$, managed by party P_i , respectively. The key distinction between Mul_GP and Mul_GS lies in whether the group element R is public or shared. These operations are performed collaboratively by N servers, where Mul_F and Mul_GS necessitate the use of Beaver multiplication triples.

F. Verification

Upon receiving the inference result shares and the proof $m\pi$ from the servers, the data owner recovers the inference result y by computing $\sum_{j=1}^N \langle y \rangle_j$.

The data owner holds the relation R_{f_cm} , the CRS, the proof $m\pi$, the statement $\{a_i\}_{i=1}^l$ which includes the input data d , the commitment cm_W , and the inference result y . $Verify(R_{f_cm}, pk, \{a_i\}_{i=1}^l, m\pi) \rightarrow \{0, 1\}$ is carried out to validate the following equation:

$$TMP = \sum_{i=0}^l a_i [\gamma^{-1} (\beta u_i(\varepsilon) + \alpha v_i(\varepsilon) + w_i(\varepsilon))]_1 \cdot [\gamma]_2 \quad (22)$$

$$[A]_1 \cdot [B]_2 = [\alpha]_1 \cdot [\beta]_2 + TMP + [C]_1 \cdot [\delta]_2 \quad (23)$$

In the event of a successful verification, the function returns 1. Otherwise, it implies that either at least one party does not follow the agreed computation protocol, or incorrect parameters are applied during the inference.

VI. SECURITY ANALYSIS

In this section, we present the security analysis of VSecNN regarding the completeness, knowledge soundness, and N -zero-knowledge properties of the multi-party proof protocol $\Gamma : (ZK_Setup, M_Prove, Verify)$.

Theorem 1: The protocol Γ is said to exhibit completeness if, for a proof $m\pi$ generated by N honest servers corresponding to $(x, w) \in R_{f_cm}$, the proof successfully convinces an honest verifier with the probability Pr :

$$Pr \left[\begin{array}{l} pk \leftarrow ZK_Setup(1^\lambda, R_{f_cm}) \\ m\pi \leftarrow M_Prove(R_{f_cm}, pk, \{\langle a_i \rangle\}_{i=1}^n) \\ 1 \leftarrow Verify(R_{f_cm}, pk, \{a_i\}_{i=1}^l, m\pi) \end{array} \right] \geq 1 - \text{negl}(\lambda) \quad (24)$$

Proof: For a proof $m\pi = ([A]_1, [B]_2, [C]_1)$ for $(x, w) \in R_{f_cm}$, it is accepted if and only if $[A]_1 \cdot [B]_2 = [\alpha]_1 \cdot [\beta]_2 + TMP + [C]_1 \cdot [\delta]_2$. We define $u(\varepsilon) = \sum_{i=0}^n a_i u_i(\varepsilon)$, $v(\varepsilon) = \sum_{i=0}^n a_i v_i(\varepsilon)$, $w(\varepsilon) = \sum_{i=0}^n a_i w_i(\varepsilon)$. We now provide the computation for both the lefthand-side (LHS) and the righthand-side (RHS) of the equation, respectively:

LHS:

$$[A]_1 \cdot [B]_2 = e(Ag, Bh) = ABe(g, h) \quad (25)$$

$$\begin{aligned} AB &= \left(\langle A \rangle_1 + \sum_{j=2}^N \langle A \rangle_j \right) \cdot \left(\langle B \rangle_1 + \sum_{j=2}^N \langle B \rangle_j \right) \\ &= \left(\alpha + \sum_{i=0}^n a_i u_i(\varepsilon) + r\delta \right) \cdot \left(\beta + \sum_{i=0}^n a_i v_i(\varepsilon) + s\delta \right) \\ &= \mathbf{u}(\varepsilon) \mathbf{v}(\varepsilon) + (\alpha\beta + \alpha v(\varepsilon) + \beta u(\varepsilon) + s\delta u(\varepsilon) + r\delta v(\varepsilon) \\ &\quad + \beta r\delta + \alpha s\delta + r s\delta^2) \end{aligned}$$

$$= \mathbf{u}(\varepsilon) \mathbf{v}(\varepsilon) + Z \quad (26)$$

RHS:

$$\begin{aligned} & [\alpha]_1 \cdot [\beta]_2 + TMP + [C]_1 \cdot [\delta]_2 \\ &= e(\alpha g, \beta h) + \sum_{i=0}^l a_i e(\gamma^{-1}(\beta u_i(\varepsilon) + \alpha v_i(\varepsilon) \\ & \quad + w_i(\varepsilon))g, \gamma h) + e(Cg, \delta h) \\ &= \left(\alpha\beta + \sum_{i=0}^l a_i(\beta u_i(\varepsilon) + \alpha v_i(\varepsilon) + w_i(\varepsilon)) + C\delta \right) e(g, h) \quad (27) \\ & \alpha\beta + \sum_{i=0}^l a_i(\beta u_i(\varepsilon) + \alpha v_i(\varepsilon) + w_i(\varepsilon)) + C\delta \\ &= \alpha\beta + \sum_{i=l+1}^n a_i(\beta u_i(\varepsilon) + \alpha v_i(\varepsilon) + w_i(\varepsilon)) + h(\varepsilon)t(\varepsilon) \\ & \quad + As\delta + Br\delta - rs\delta^2 \\ &= \mathbf{w}(\varepsilon) + \mathbf{h}(\varepsilon)\mathbf{t}(\varepsilon) + (\alpha\beta + \alpha v(\varepsilon) + \beta u(\varepsilon) + s\delta u(\varepsilon) \\ & \quad + r\delta v(\varepsilon) + \beta r\delta + \alpha s\delta + rs\delta^2) \\ &= \mathbf{w}(\varepsilon) + \mathbf{h}(\varepsilon)\mathbf{t}(\varepsilon) + Z \quad (28) \end{aligned}$$

Thus, if the verifier outputs 0 for a correct proof $m\pi$, it implies that $u(\varepsilon)v(\varepsilon) \neq w(\varepsilon) + h(\varepsilon)t(\varepsilon)$, which contradicts the equation (3) in Section III-C for the QAP relation R_{f_cm} . Therefore, the protocol Γ exhibits completeness.

Theorem 2: The protocol Γ is said to exhibit knowledge soundness if, for any $t \leq N$ computationally bounded adversaries $\mathcal{A}_i$ and a computationally bounded extractor $\mathcal{E}$ with complete access to the states of the t adversaries, the following condition is met:

$$Pr \left[\begin{array}{l} pk \leftarrow ZK_Setup(1^\lambda, R_{f_cm}) \\ (x, m\pi', w) \leftarrow (\mathcal{A}|\mathcal{E})(R_{f_cm}, pk) : \\ (x, w) \notin R_{f_cm} \wedge \\ 1 \leftarrow \text{Verify}(R_{f_cm}, pk, \{a_i\}_{i=1}^l, m\pi') \end{array} \right] \leq \text{negl}(\lambda) \quad (29)$$

Proof: Assume the existence of a simulator that using the trapdoor $sk = (\alpha, \beta, \gamma, \delta, \varepsilon)$ from the trusted setup phase, can compute a simulated proof $m\pi$. Specifically, the simulator randomly generates $A, B \in \mathbb{Z}_p$ and then computes:

$$C = \frac{A \cdot B - \alpha\beta - \sum_{i=0}^l a_i(\beta u_i(\varepsilon) + \alpha v_i(\varepsilon) + w_i(\varepsilon))}{\delta} \quad (30)$$

This simulated proof, denote as $m\pi = ([A]_1, [B]_2, [C]_1)$, can convince the verifier without the need for a witness.

If an extractor $\mathcal{E}$ can compute a witness w for the collaboratively created proof $m\pi'$ by the t adversaries and the $N - t$ honest servers, then $m\pi'$ and the simulated proof $m\pi$ must be distinguishable. This implies that if the verifier approves $m\pi$, it must reject $m\pi'$.

Consequently, the extractor $\mathcal{E}$ cannot determine a witness w , such that $(x, w) \in R_{f_cm}$, for an accepted proof $m\pi'$, which is generated by adversaries who are unaware of the witness. The probability Pr is negligible and the protocol Γ exhibits knowledge soundness.

Theorem 3: The protocol Γ is said to exhibit N-zero-knowledge if a computationally bounded adversary $\mathcal{A}$ cannot

derive any information about the secret shares of the witness from the N proof shares generated by N servers. Formally, for all $(x, w) \in R_{f_cm}$ and a computationally bounded adversary $\mathcal{A}$, the following condition is satisfied:

$$\begin{aligned} & Pr \left[\begin{array}{l} pk \leftarrow ZK_Setup(1^\lambda, R_{f_cm}) \\ \langle m\pi \rangle_j \leftarrow M_Prove(R_{f_cm}, pk, \{\langle a_i \rangle_{j=1}^n\}) : \\ 1 \leftarrow \mathcal{A}(R_{f_cm}, pk, sk, m\pi) \end{array} \right] \\ &= Pr \left[\begin{array}{l} pk \leftarrow ZK_Setup(1^\lambda, R_{f_cm}) \\ \langle m\pi \rangle_j \leftarrow Sim(R_{f_cm}, sk, \{\langle a_i \rangle_{j=1}^l\}) : \\ 1 \leftarrow \mathcal{A}(R_{f_cm}, pk, sk, m\pi) \end{array} \right] \quad (31) \end{aligned}$$

Proof: Assume the existence of a set of N simulators. Each simulator is capable of using the trapdoor $sk = (\alpha, \beta, \gamma, \delta, \varepsilon)$, to compute a simulated proof share $\{\langle m\pi \rangle_j\}_{j=1}^N$ by executing $\langle m\pi \rangle_j \leftarrow Sim(R_{f_cm}, sk, \{\langle a_i \rangle_{j=1}^l\})$ as follows:

Each simulator $\mathcal{B}_j$ randomly generates $\langle A \rangle_j, \langle B \rangle_j \in \mathbb{Z}_p$ where $A = \sum_{j=1}^N \langle A \rangle_j, B = \sum_{j=1}^N \langle B \rangle_j$. They then compute:

$$\begin{aligned} & \langle C \rangle_j \\ &= \frac{\langle A \rangle_j \cdot \langle B \rangle_j - \alpha\beta - \sum_{i=0}^l \langle a_i \rangle_j (\beta u_i(\varepsilon) + \alpha v_i(\varepsilon) + w_i(\varepsilon))}{\delta} \quad (32) \end{aligned}$$

Thus, simulators $\mathcal{B}_j$ can generate simulated proof shares $m\pi_j = ([\langle A \rangle_j]_1, [\langle B \rangle_j]_2, [\langle C \rangle_j]_1)$ and aggregated simulated proof $m\pi = \sum_{j=1}^N \langle m\pi \rangle_j = ([A]_1, [B]_2, [C]_1)$ without the witness. This simulated proof also convince the verifier.

Considering the basic Groth16 protocol as Γ_G , which is established to be zero-knowledge, we introduce a simulator $\mathcal{B}$ with complete access to the state of an adversary $\mathcal{A}$, to distinguish between the actual proof and the simulated proof in Groth16. If the adversary $\mathcal{A}$ can derive the witness from the proof shares $\langle m\pi \rangle_j$ computed using the witness $\{\langle a_i \rangle_{j=l+1}^n\}$, then both the actual proof shares and the simulated proof shares would be distinguishable. The simulator $\mathcal{B}$ would then leverage the adversary $\mathcal{A}$ to discern between the actual and the simulated proof in Groth16.

Therefore, if the Groth16 protocol is zero-knowledge, the simulated and actual proof shares in our protocol Γ must be indistinguishable. Consequently, a computationally bounded adversary $\mathcal{A}$ cannot obtain any information about the secret shares of the witness from the N proof shares generated by N servers. The protocol Γ exhibits N -zero-knowledge.

VII. EXPERIMENTS

A. Experimental Settings

1) **Implementation:** All experiments are conducted on either one or two desktop machines. These machines run on Ubuntu 20.04 and are equipped with an Intel Core i7-10700k @ 3.8GHz CPU and 128GB of RAM. Key aspects to be considered during the implementation of VSecNN:

- While neural networks extensively use floating-point operations, MPC and zk-SNARKs that inherently work with finite fields and elliptic curve groups, are favoring fix-point arithmetic. Given this incompatibility, we convert the floating-point model trained using PyTorch into a fixed-point model using the quantization method [48].

TABLE II
DETAILS OF THE DATASETS

Dataset	Categories	Features	Samples
WINE	3	13	178
MNIST	10	$28 \times 28 \times 1$	70,000
CIFAR10	10	$32 \times 32 \times 3$	60,000
ORL	40	$46 \times 56 \times 3$	400

TABLE III
MODEL ARCHITECTURES

Layer	WINE	MNIST	CIFAR10	ORL
Con1 ^a	/	/	K: 5×5 , C: 6 ^c	K: 5×5 , C: 6
Avg.Pooling	/	/	K: 2×2	K: 2×2
Con2	/	/	K: 5×5 , C: 16	K: 5×5 , C: 16
Avg.Pooling	/	/	K: 2×2	K: 2×2
Con3	/	/	K: 4×4 , C: 120	K: 4×4 , C: 120
FC1 ^b	13×32	784×128	480×84	4800×84
FC2	32×3	128×10	84×10	84×40

^a Conx represents the x-th convolutional layer

^b FCx represents the x-th fully connected layer

^c K represents the kernel size, C represents the number of out channels

- The fundamental Groth16 algorithm is implemented through the arkworks library in RUST. We utilized the mpc-net [30] for network communication. We opt for the BLS12-381, a pairing-friendly curve which provides a 128-bit (i.e., $\lambda = 128$) security level, for Gorth16 and MPC protocol over elliptic curve groups.
- The arithmetic secret sharing protocol is selected as the underlying MPC protocol, and five operations (equation (17- 21)) are implemented using RUST for 1) enabling multi-party inference with the quantized neural network model and 2) facilitating the multi-party proof generation inherent in Groth16. It's worth noting that this approach can accommodate to other MPC protocols designed for multi-party inference, such as SecureML [3], Cheetah [4].
- We convert the quantized neural network model into a QAP relation using the arkworks library, and then apply this relation to the multi-party proof generation process implemented earlier to achieve verifiable multi-party neural network inference.

2) *Datasets & Models*: The experiments are conducted on four datasets involving classification tasks of varying scales: Wine Data Set (WINE), Modified National Institute of Standards and Technology database (MNIST), Canadian Institute for Advanced Research, 10 classes (CIFAR-10), and Our Database of Faces (ORL), respectively. Accordingly, we consider four distinct neural network architectures, with varying orders of magnitude regarding parameters for inference, on the aforementioned datasets. Particularly, fully connected neural networks (ShallowNet) are used for WINE and MNIST, denoted by WINE and MNIST for simplicity, while variants of LeNet are adopted for CIFAR10 and ORL, denoted by CIFAR10 and ORL. Detailed information about the datasets and model architectures are shown in TABLE II and TABLE III.

B. Experimental Results

Our work extends existing multi-party secure inference approaches by providing additional verification. As a result, our experiments primarily focus on the generating and verifying multi-party proofs, which constitute the essence of VSecNN.

1) *Comparisons on Proof Generation Time and Verification Time*: VSecNN(2) refers to our scheme involving two servers, while VNN(1) represents a single-party verifiable neural network inference scheme implemented under our VSecNN benchmark. In fact, VNN(1) also corresponds to the implementation of the proof of accuracy component in our scheme, thereby reflecting the performance of our approach in proving model authenticity. Unlike VSecNN, VNN(1) does not provide privacy preservation for the inferred data and inference results. VSecNN(2) runs on two desktop machines connected via a local area network (LAN). We compare VSecNN(2), VNN(1), and ZEN [25], which achieves verifiable neural network inference using Groth16. However, similar to VNN(1), ZEN conducts inference and proof generation within a single party, lacking privacy preservation for inferred data and results. Our goal is to show that our multi-party solution retains computational efficiency on par with single-party alternatives while also ensuring data privacy. As illustrated in TABLE IV, our comparisons across various datasets and models demonstrate that our method surpasses ZEN in most cases, particularly in terms of proof generation time. This advantage stems from the optimizations in developing the VSecNN, such as transforming the commonly used finite field-based MPC protocol to the elliptic curve groups-based protocol for accommodating zk-SNARKs. In addition, the KZG polynomial commitment is a better alternative for neural networks than the Pedersen commitment adopted in ZEN. Furthermore, the verification time for proofs generated by multiple servers in VSecNN aligns with those produced in single-party context. This highlights the user-friendliness of our approach, as the proof generation process is managed by the cloud server, while verification process is carried out by the user.

2) *Impact of Number of Servers*: We further explore the impacts of number of servers on the generation time for the proofs. As presented in TABLE V, the findings suggest that an increase in the number of servers has negligible influence on the proof generation time. However, as the number of servers increases, there is a notable rise in communication overhead. This is because prior to the proof generation phase, the servers must collaborate to establish a constraint system, where communication costs become the predominant expense. It implies that while the capabilities of resisting collusion attacks can be improved by increased number of servers, this enhancement trades off against a higher resource requirement. Hence, a key takeaway for researchers would be that future studies ought to focus on creating a state-of-the-art design that effectively addresses this balance.

3) *Number of Constraints*: Constraints are key components in computation complexity, defining how inputs, outputs, and intermediate results are interrelated, while ensuring these relationships are preserved in the proof. Accordingly, the

TABLE IV
COMPARISON ON PROOF GENERATION TIME AND VERIFICATION TIME

Model	Generation (s)			Verification (s)		
	ZEN	VNN(1)	VSecNN(2) ^a	ZEN	VNN(1)	VSecNN(2)
WINE	36.560	2.056	3.420	0.084	0.071	0.070
MNIST	817.010	108.590	227.044	0.386	0.165	0.169
CIFAR10	714.381 ^b	733.506	1401.045	0.317	7.129	7.057
ORL	6,554.729	2,752.191	4882.097	1.444	24.296	24.451

^a The number of servers is indicated within the parentheses.

^b Although the CIFAR-10 model has more constraints, this triggers the SIMD-style optimization in the ZEN scheme, leading to a shorter proof generation time compared to the MNIST model.

TABLE V
IMPACT OF NUMBER OF SERVERS

Model	WINE		MNIST		CIFAR10		ORL	
	Generation(s)	Communication	Generation(s)	Communication	Generation(s)	Communication	Generation(s)	Communication
VSecNN (2)	3.420	0.619 MB	227.044	124.038 MB	1401.045	0.901 GB	4882.097	3.407 GB
VSecNN (3)	3.484	1.856 MB	227.902	372.115 MB	1442.405	2.704 GB	4903.579	10.220 GB
VSecNN (4)	3.478	3.713 MB	223.932	744.231 MB	1411.739	5.407 GB	4895.861	20.439 GB
VSecNN (5)	3.558	6.188 MB	228.275	1240.384 MB	1417.225	9.012 GB	4901.937	34.065 GB

TABLE VI
NUMBER OF CONSTRAINTS(K)

Layer	WINE	MNIST	CIFAR10	ORL
Con1	/	/	1415.904	1323.504
ReLU	/	/	9.408	26.208
Avg.Pooling	/	/	3.528	9.828
Con2	/	/	961.600	3596.384
ReLU	/	/	3.200	11.968
Avg.Pooling	/	/	1.200	4.224
Con3	/	/	492.000	4920.000
FC1	1.696	401.536	161.364	1612.884
ReLU	0.064	0.256	0.168	0.168
FC2	0.387	5.130	3.370	13.480

TABLE VII
COMPARISON OF ZK-SNARKS IN MPC SETTINGS

Scheme	Generation(ms)	Verification(ms)	Proof Size(B)
MPC+Groth16 (Ours)	94.033	46.805	432
MPC+Marlin	339.839	125.550	2392
MPC+Plonk	309.138	268.720	4640

efficiency of proof generation depends on both the number of constraints and the computation cost of generating proofs for each constraint. We examine the number of constraints in each layer of the neural network across various models and datasets. First, we present the number of constraints required for each neural networks layer. As shown in TABLE VI, the results lay the foundation for future research on optimizing constraint quantities, particularly for convolutional layers. For instance, in LeNet for CIFAR10, the first convolutional layer (Con1) requires 1415.904K constraints, whereas the subsequent pooling layer only requires 3.528K constraints. Furthermore, we also observe that the number of constraints is predominantly influenced by the underlying computational circuits and the approach to constraint construction, thus maintaining the status quo with a varying number of servers.

4) *Comparison of Zk-SNARKs in Mpc Settings*: To further evaluate the efficiency of our design, we conducted an additional experiment comparing different zk-SNARK protocols

TABLE VIII
CRS AND PROOF SIZE OF LENET-CIFAR10

Scheme	CRS Size	Proof Size
vCNN	40.07 MB	2803 B
pvCNN	14.30 TB	343.19 MB
VsecNN	5.48 GB	432 B

when integrated with MPC. Specifically, we refer to the scheme in [30], which demonstrates the combination of MPC with various zk-SNARK protocols, though without applying it to neural network inference. Following this setting, we benchmarked a basic multiplication circuit executed in a multi-party environment, measuring the proof generation time, verification time, and proof size under three configurations: MPC+Groth16 (our choice), MPC+Marlin, and MPC+Plonk. The experimental results, presented in TABLE VII, show that MPC+Groth16 achieves the fastest proof generation and verification times, while also producing significantly smaller proofs compared to MPC+Marlin and MPC+Plonk. While Marlin and Plonk feature universal, circuit-independent public parameters and can operate without a trusted setup, the MPC+Groth16 combination remains more favorable in terms of proof generation time, verification time, and proof size in our setting.

5) *CRS and Proof Sizes*: We compare the CRS and proof sizes among VSecNN, vCNN [6], and pvCNN [39]. vCNN is a verifiable CNN inference scheme based on a zk-SNARK protocol using quadratic polynomial program (QPP), but lacks data privacy preservation for data inferred. pvCNN offers a privacy-preserving and verifiable CNN collaborative inference scheme, however, it only secures a portion of the model and data, with proof generation limited to parts of the model. Their partial implementations present a challenge for a direct comparison with our full implementation. As a result, we focus on comparing the sizes of CRS and proof, which remain independent of the implementation methods and testing

environments. As shown in TABLE VIII, our tests, conducted on LeNet-CIFAR10 inference, demonstrate that our scheme substantially improves proof size. While the size of the CRS in vCNN is considerably smaller than ours, it only achieves single-party proof generation and fails to ensure the privacy preservation of the inferred data and inference results.

VIII. CONCLUSION

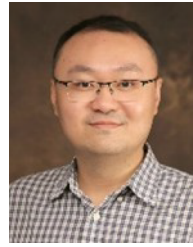
In this study, we simultaneously address the challenges of verifiability and privacy preservation in neural network inference. We develop a multi-party proof generation approach by leveraging MPC and Groth16. We also optimize the multi-party proof generation approach to accommodate zk-SNARKs over elliptic curve groups. We then introduce VSecNN by incorporating the multi-party proof generation approach to provide a solution for achieving verifiable and privacy-preserving neural network inference. Finally, we conduct extensive experiments to evaluate the performance of VSecNN. We demonstrate that significant efficiency can be achieved through VSecNN during the generating and verifying multi-party proofs.

Our research highlights the benefits of integrating zk-SNARKs with multi-party neural network inference to alleviate the concerns about verifiability. However, this approach may potentially require a substantial computational time and result in a comparatively large CRS size. Future research may focus on how to extend our VSecNN to a resource-limited context by optimizing the parameters such as the number of constraints and the size of the CRS. Moreover, we aim to explore the extension of VSecNN to support large-scale models such as transformer-based architectures. This requires reducing the circuit complexity to mitigate the overhead of proof generation. Potential optimizations include batching repetitive operations (e.g., scalar multiplications) into parallelizable components to improve prover efficiency.

REFERENCES

- [1] J. Liu et al., "Privacy-preserving and verifiable outsourcing linear inference computing framework," *IEEE Trans. Services Comput.*, vol. 16, no. 6, pp. 4591–4604, Nov. 2023.
- [2] A. Yang, J. Xu, J. Weng, J. Zhou, and D. S. Wong, "Lightweight and privacy-preserving delegatable proofs of storage with data dynamics in cloud storage," *IEEE Trans. Cloud Comput.*, vol. 9, no. 1, pp. 212–225, Jan. 2021.
- [3] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 19–38.
- [4] Z. Huang, W. J. Lu, C. Hong, and J. Ding, "Cheetah: Lean and fast secure two-party deep neural network inference," in *Proc. 31st USENIX Secur. Symp. (USENIX Security)*, Boston, MA, USA, Aug. 2022, pp. 809–826.
- [5] N. Jawalkar, K. Gupta, A. Basu, N. Chandran, D. Gupta, and R. Sharma, "Orca: FSS-based secure training and inference with GPUs," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 597–616.
- [6] S. Lee, H. Ko, J. Kim, and H. Oh, "VCNN: Verifiable convolutional neural network based on zk-SNARKs," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 4254–4270, Jul. 2024.
- [7] B.-J. Chen, S. Waiwitlikhit, I. Stoica, and D. Kang, "ZKML: An optimizing system for ML inference in zero-knowledge proofs," in *Proc. 19th Eur. Conf. Comput. Syst.*, Apr. 2024, pp. 560–574.
- [8] H. Sun, J. Li, and H. Zhang, "ZkLLM: Zero knowledge proofs for large language models," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 4405–4419.
- [9] C. Weng, K. Yang, X. Xie, J. Katz, and X. Wang, "Mystique: Efficient conversions for zero-knowledge proofs with applications to machine learning," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, 2021, pp. 501–518.
- [10] Z. Peng et al., "A survey of zero-knowledge proof based verifiable machine learning," 2025, *arXiv:2502.18535*.
- [11] Z. Xing et al., "Zero-knowledge proof-based verifiable decentralized machine learning in communication network: A comprehensive survey," 2023, *arXiv:2310.14848*.
- [12] S. R. Rachuri, A. Suresh, and H. Chaudhari, "Trident: Efficient 4PC framework for privacy preserving machine learning," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, Jul. 2020, pp. 1–18.
- [13] S. Wagh, S. Tople, F. Benhamouda, E. Kushilevitz, P. Mittal, and T. Rabin, "FALCON: Honest-majority maliciously secure framework for private deep learning," in *Proc. Priv. Enhancing Technol. Symp.*, Jul. 2021, pp. 188–208.
- [14] A. Dalskov, D. Escudero, and M. Keller, "Fantastic four: Honest-majority four-party secure computation with malicious security," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, 2021, pp. 2183–2200.
- [15] B. Yuan, S. Yang, Y. Zhang, N. Ding, D. Gu, and S.-F. Sun, "MD-ML: Super fast privacy-preserving machine learning for malicious security with a dishonest majority," in *Proc. 33rd USENIX Secur. Symp. (USENIX Secur.)*, 2024, pp. 2227–2244.
- [16] C. Dong et al., "Fusion: Efficient and secure inference resilient to malicious servers," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.
- [17] G. Xu et al., "Secure and verifiable inference in deep neural networks," in *Proc. Annu. Comput. Secur. Appl. Conf. (ACSAC)*, Austin, TX, USA, 2020, pp. 784–797.
- [18] X. Li, J. He, P. Vijayakumar, X. Zhang, and V. Chang, "A verifiable privacy-preserving machine learning prediction scheme for edge-enhanced HCPSS," *IEEE Trans. Ind. Informat.*, vol. 18, no. 8, pp. 5494–5503, Aug. 2022.
- [19] C. Niu, F. Wu, S. Tang, S. Ma, and G. Chen, "Toward verifiable and privacy preserving machine learning prediction," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1703–1721, May 2022.
- [20] W. Cao, W. Shen, J. Qin, and H. Lin, "Privacy-preserving and verifiable convolution neural network inference and training in cloud computing," *Future Gener. Comput. Syst.*, vol. 164, Mar. 2025, Art. no. 107560.
- [21] J. Cai, W. Shen, and J. Qin, "ESVFL: Efficient and secure verifiable federated learning with privacy-preserving," *Inf. Fusion*, vol. 109, Sep. 2024, Art. no. 102420.
- [22] Z. Ghodsi, T. Gu, and S. Garg, "SafetyNets: Verifiable execution of deep neural networks on an untrusted cloud," in *Proc. 31st Conf. Neural Inf. Process. Syst. (NIPS)*, vol. 30, 2017, pp. 4675–4684.
- [23] T. Liu, X. Xie, and Y. Zhang, "ZkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.* New York, NY, USA: ACM, Nov. 2021, pp. 2968–2985.
- [24] T. Lu et al., "An efficient and extensible zero-knowledge proof framework for neural networks," *Cryptol. ePrint Arch.*, 2024. [Online]. Available: <https://eprint.iacr.org/2024/703>
- [25] B. Feng, L. Qin, Z. Zhang, Y. Ding, and S. Chu, "ZEN: An optimizing compiler for verifiable, zero-knowledge neural network inferences," *Cryptol. ePrint Arch.*, 2021.
- [26] Y. Fan et al., "PsvCNN: A zero-knowledge CNN prediction integrity verification strategy," *IEEE Trans. Cloud Comput.*, vol. 12, no. 2, pp. 359–369, Apr. 2024.
- [27] Y. Fan, K. Ma, L. Zhang, X. Lei, G. Xu, and G. Tan, "ValidCNN: A large-scale CNN predictive integrity verification scheme based on zk-SNARK," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 6, pp. 5185–5195, Nov. 2024.
- [28] P. Mohassel and P. Rindal, "ABY3: A mixed protocol framework for machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2018, pp. 35–52.
- [29] S. Kanjalkar, Y. Zhang, S. Gandlur, and A. Miller, "Publicly auditable MPC-as-a-service with succinct verification and universal setup," in *Proc. IEEE Eur. Symp. Secur. Privacy Workshops (EuroSPW)*, Sep. 2021, pp. 386–411.
- [30] A. Ozdemir and D. Boneh, "Experimenting with collaborative zk-SNARKs: Zero-knowledge proofs for distributed secrets," in *Proc. 31st USENIX Secur. Symp. (USENIX Security)*, Boston, MA, USA, Aug. 2022, pp. 4291–4308.
- [31] J. Groth, "On the size of pairing-based non-interactive arguments," in *Proc. 35th Annu. Int. Conf. Theory Appl. Cryptograph. Techn.*, Aug. 2016, pp. 305–326.

- [32] F. Bourse, M. Minelli, M. Minihold, and P. Paillier, "Fast homomorphic evaluation of deep discretized neural networks," in *Proc. Annu. Int. Cryptol. Conf. (CRYPTO)*. Cham, Switzerland: Springer, 2018, pp. 483–512.
- [33] D. Kim and C. Guyot, "Optimized privacy-preserving CNN inference with fully homomorphic encryption," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2175–2187, 2023.
- [34] J. Liu, M. Juuti, Y. Lu, and N. Asokan, "Oblivious neural network predictions via minion transformations," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 619–631.
- [35] W. Z. Srinivasan, P. Akshayaram, and P. R. Ada, "DELPHI: A cryptographic inference service for neural networks," in *Proc. 29th USENIX Secur. Symp. (USENIX Secur.)*, Aug. 2019, pp. 2505–2522.
- [36] R. Lehmkuhl, P. Mishra, A. Srinivasan, and R. A. Popa, "Muse: Secure inference resilient to malicious clients," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, 2021, pp. 2201–2218.
- [37] N. Chandran, D. Gupta, S. L. B. Obbattu, and A. Shah, "SIMC: ML inference secure against malicious clients at semi-honest cost," in *Proc. 31st USENIX Secur. Symp. (USENIX Secur.)*, Jun. 2022, pp. 1361–1378.
- [38] X. Hou et al., "CipherGPT: Secure two-party GPT inference," *Cryptol. ePrint Arch.*, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1147>
- [39] J. Weng, J. Weng, G. Tang, A. Yang, M. Li, and J.-N. Liu, "PvCNN: Privacy-preserving and verifiable convolutional neural network testing," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2218–2233, 2023.
- [40] A. Riasi, J. Guajardo, and T. Hoang, "Privacy-preserving verifiable neural network inference service," 2024, *arXiv:2411.07468*.
- [41] M. Li et al., "Accurate, secure, and efficient semi-constrained navigation over encrypted city maps," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2642–2658, May 2025.
- [42] I. Damgård, V. Pastro, N. P. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *Proc. Cryptol. Conf. (CRYPTO)*, 2012, pp. 643–662.
- [43] D. Demmler, T. Schneider, and M. Zohner, "ABY—A framework for efficient mixed-protocol secure two-party computation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2015, pp. 1–15.
- [44] Y. Dong et al., "PUMA: Secure inference of LLaMA-7B in five minutes," 2023, *arXiv:2307.12533*.
- [45] A. Kate, G. M. Zaverucha, and I. Goldberg, "Constant-size commitments to polynomials and their applications," in *Proc. 16th Int. Conf. Theory Appl. Cryptol. Inf. Security*, Singapore, Dec. 2010, pp. 177–194.
- [46] M. Li et al., "Astraea: Anonymous and secure auditing based on private smart contracts for donation systems," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 3002–3018, Jul. 2023.
- [47] M. Li, Y. Chen, C. Lal, M. Conti, F. Martinelli, and M. Alazab, "Nereus: Anonymous and secure ride-hailing service based on private smart contracts," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 2849–2866, Jul. 2023.
- [48] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2704–2713.



Meng Li (Senior Member, IEEE) received the Ph.D. degree in computer science and technology from the School of Computer Science and Technology, Beijing Institute of Technology (BIT), China, in 2019. He was a Post-Doctoral Researcher with the Department of Mathematics and the HIT Center, University of Padua, Italy, where he is currently with the Security and PRIVacy Through Zeal (SPRITZ) Research Group led by Prof. Mauro Conti (Fellow, IEEE). He is an Associate Professor and a Personnel Secretary with the School of Computer Science and Information Engineering, Hefei University of Technology (HFUT), China. He was sponsored by ERCIM 'Alain Bensoussan' Fellowship Program to conduct post-doctoral research supervised by Prof. Fabio Martinelli at CNR, Italy, from January 2020 to March 2021. He was sponsored by China Scholarship Council (CSC) as a Joint Ph.D. Student supervised by Prof. Xiaodong Lin (Fellow, IEEE) in the Broadband Communications Research (BBCR) Laboratory, University of Waterloo, and Wilfrid Laurier University, Canada, from September 2017 to August 2018. He was supported by CSC as a Visiting Scholar collaborating with Prof. Mauro Conti (Fellow, IEEE) at the HIT Center, University of Padua, from March 2025 to June 2025. His research interests include security, privacy, applied cryptography, blockchain, TEE, and the Internet of Vehicles. In this area, he has published 129 papers in topmost journals and conferences, including IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY (IEEE TIFS), IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING (IEEE TDSC), IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, TODS, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, IEEE TRANSACTIONS ON SERVICES COMPUTING, COMST, IEEE S&P, USENIX Security, ACM MobiCom, and ISSTA. He is a Senior Member of CIE, CIC, and CCF. He served as a TPC Member for conferences, including ICDCS, Inscrypt, ICICS, and TrustCom. He was a recipient of the 2024 IEEE HITC Award for Excellence (Early Career Researcher) and the 2025 IEEE TCSVC Rising Star Award. He is an Associate Editor of IEEE TIFS, IEEE TDSC, and IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT.



Zijian Zhang (Senior Member, IEEE) received the Ph.D. degree from the School of Computer Science and Technology, Beijing Institute of Technology. He was a Visiting Scholar with the Computer Science and Engineering Department, State University of New York at Buffalo, in 2015. He is currently a Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. His research interests include design of authentication and key agreement protocol and analysis of entity behavior and preference.



Wenti Yang received the Ph.D. degree in information security from North China Electric Power University, Beijing, China, in 2024. She is currently an Assistant Researcher with the Department of New Networks, Peng Cheng Laboratory (PCL), Shenzhen, Guangdong, China. Her research interests include privacy-preserving machine learning and security in computing power networks.



Zhitao Guan (Member, IEEE) received the B.Eng. and Ph.D. degrees in computer application from Beijing Institute of Technology, China, in 2002 and 2008, respectively. He is currently a Professor with the School of Control and Computer Engineering, North China Electric Power University. His current research interests include smart grid security, AI security, system security, and privacy computing. He has authored over 100 peer-reviewed journal and conference papers in these areas.



Xuan Li received the B.E. degree in information security from the School of Control and Computer Engineering, North China Electric Power University, Beijing, China, in 2022. He is currently working toward the M.S. degree with North China Electric Power University. His research interests include privacy computing and federated learning.



Liehuang Zhu (Senior Member, IEEE) received the M.S. degree in computer science from Wuhan University, Wuhan, China, in 2001, and the Ph.D. degree in computer science from Beijing Institute of Technology, Beijing, China, in 2004. He is currently a Full Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. His research interests include data security and privacy protection, blockchain applications, and AI security. He has authored more than 500 journal and conference papers in these areas.